pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Organizing a Small Portfolio

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

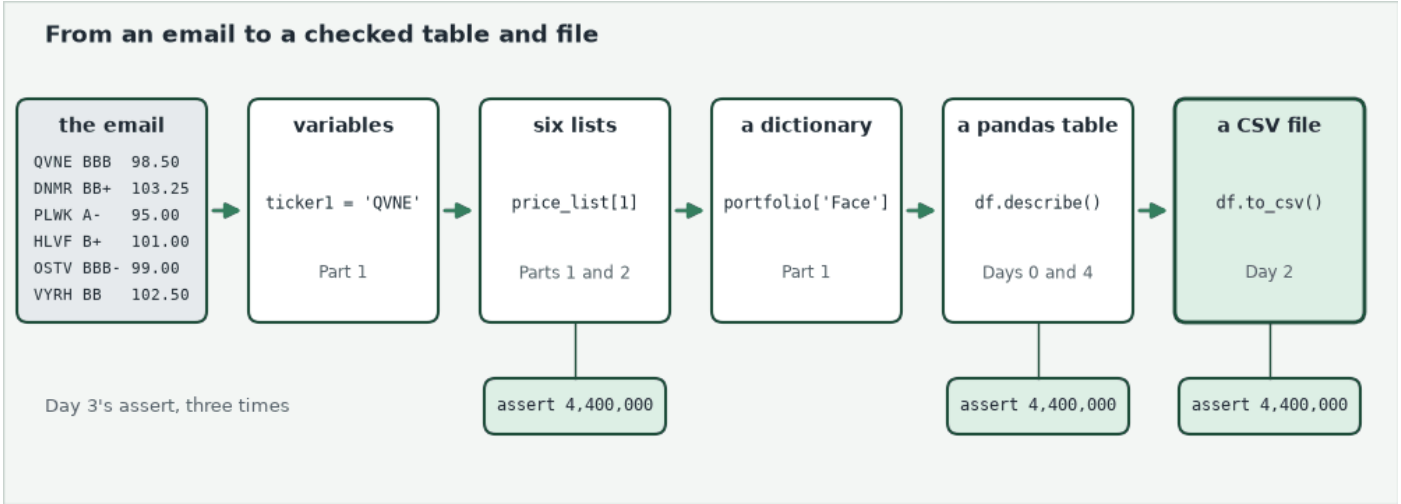
This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem definition: the seventh note

Another team has handed us OSTV and VYRH. Their email lists all six of our bonds, typed into the body: no file, no system record. Put the book in order before anyone analyzes it, make sure it ties to the email's 4,400,000 face, and hand it on as a file.

Objective: store the six bonds to

1. Look up everything about one bond
2. Update a bond when its rating or price changes
3. Answer simple questions about the book, and
4. Hand it on as a table built for analysis, and as a file



The data: six bonds pasted at A1

A: Ticker	B: Issuer	C: Sector	D: Rating	E: Score	F: Coupon (%)	G: Price	H: Face held
QVNE	Quorvane Energy	Energy	BBB	9	5.25	98.50	1,500,000
DNMR	Dunmarrow Rail	Transportation	BB+	11	6.125	103.25	500,000
PLWK	Pellwick Paper	Materials	A-	7	4.75	95.00	750,000
HLVF	Halvering Foods	Consumer	B+	14	7.50	101.00	250,000
OSTV	Ostrevale Telecom	Telecom	BBB-	10	5.75	99.00	1,000,000
VYRH	Veyrholt Health	Health Care	BB	12	6.75	102.50	400,000

5.1 One variable per ticker

```
print(ticker1, ticker2, ticker3, ticker4, ticker5, ticker6)
QVNE DNMR PLWK HLVF OSTV VYRH
```

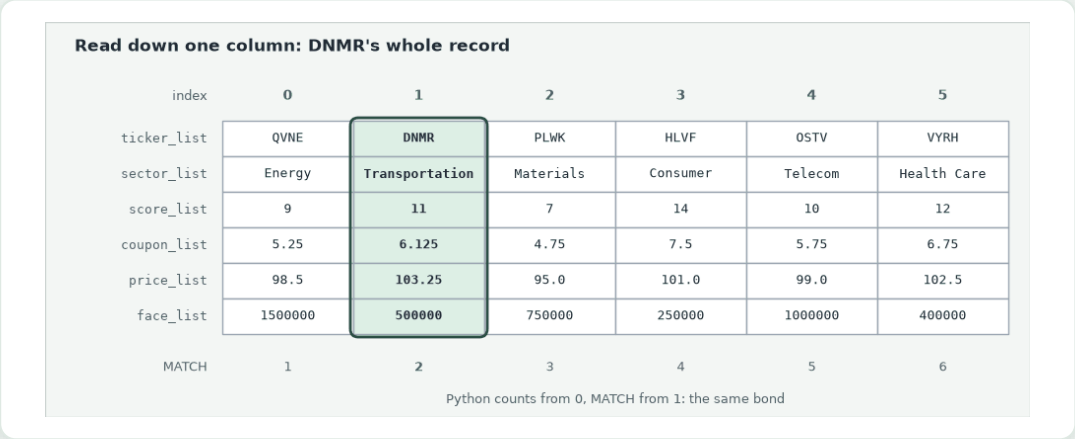
One variable per value does not scale. Store each column as a list.

5.2 Six lists: tie them, then find a bond

Tie the lists to the email

```
assert len(face_list) == len(ticker_list), 'the face amounts and tickers do not match'
total_face_before = sum(face_list)
assert total_face_before == 4400000, 'the face does not tie to the email'
print(f'{len(ticker_list)} bonds with a total face of {total_face_before:,}')
6 bonds with a total face of 4,400,000
```

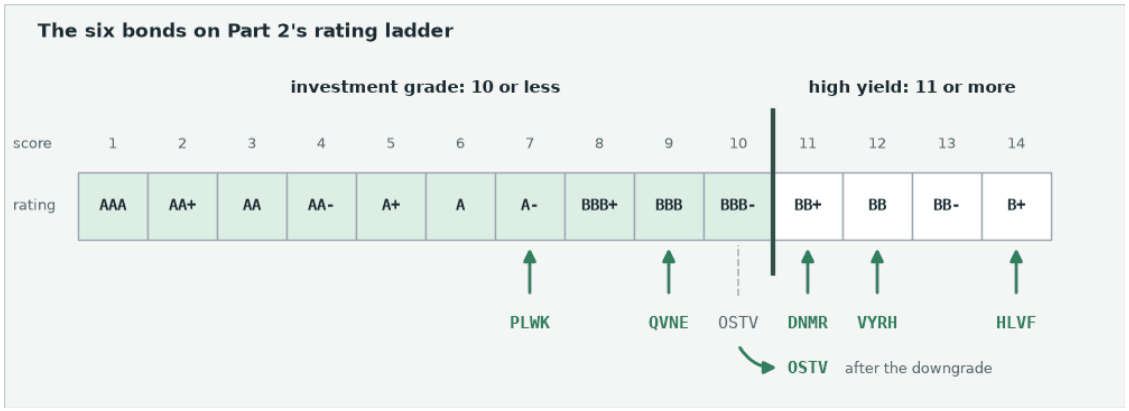
Excel side by side. The check cell =IF(SUM(H2:H7)=4400000,"OK","CHECK") shows OK, and =COUNTA(A2:A7)=COUNT(H2:H7) shows TRUE.




```
pos = ticker_list.index('DNMR')
print(pos)
1
print(sector_list[pos])
print(score_list[pos])
print(coupon_list[pos])
Transportation
11
6.125
```

Excel side by side. =MATCH("DNMR",A2:A7,0) finds the position and returns 2; =INDEX(F2:F7,2) reads the coupon there, 6.125, as coupon_list[pos] does.

A downgrade, then a correction for high yield




```
pos = ticker_list.index('OSTV')
score_list[pos] = 11
print(score_list)
[9, 11, 7, 14, 11, 12]
```

OSTV has crossed from investment grade to high yield.

Every bond in one loop: half a point off high yield

```
price_list = email_price_list.copy()
for i, score in enumerate(score_list):
    if score >= 11:
        price_list[i] = price_list[i] - 0.5
        print('Corrected', ticker_list[i])
print(price_list)
Corrected DNMR
Corrected HLVF
Corrected OSTV
Corrected VYRH
[98.5, 102.75, 95.0, 100.5, 98.5, 102.0]
```

Excel side by side

With E6 updated to 11, the correction goes in a new column I: =IF(E2>=11,G2-0.5,G2), filled down. The loop changes the prices where they sit.

Count by a condition

```
count = 0
for score in score_list:
    if score <= 10:
        count += 1
print('Investment grade bonds:', count)
Investment grade bonds: 2
```

```
investment_grade = 0
high_yield = 0
for score in score_list:
    if score <= 10:
        investment_grade += 1
    else:
        high_yield += 1
print('Investment grade:', investment_grade)
print('High yield:', high_yield)
Investment grade: 2
High yield: 4
```

Excel side by side

=COUNTIF(E2:E7,"<=10") returns 2 and =COUNTIF(I2:I7,">100") returns 3. A loop can also do more to each matching bond while it is there.

The same lines would correct 200 bonds. Now ask the book questions.

Predict first: decide, then run it in the notebook

Predict 1

Each bond as one record: "bond_a" in square brackets, "bond_b" in round. Both are marked up a quarter point. What happens?

```
bond_a = ['QVNE', 9, 98.5]
bond_b = ('DNMR', 11, 103.25)
bond_a[2] = 98.75
print(bond_a)
bond_b[2] = 103.5
print(bond_b)
```

Predict 2

Count the bonds that are high yield and also above par: "and" or "or" between the tests, and what count? The cell tries both.

```
count_and = 0
count_or = 0
for score, price in zip(score_list, price_list):
    if score >= 11 and price > 100:
        count_and += 1
    if score >= 11 or price > 100:
        count_or += 1
print('With and:', count_and)
print('With or:', count_or)
```

Predict 3

Three trades wait to settle. How many times does "Settling a trade" print, and when does the last line run?

```
trades_to_settle = 3
while trades_to_settle > 0:
    print('Settling a trade')
    trades_to_settle = trades_to_settle - 1
print('All trades settled')
```

5.4 A function for any bond

```
def bond_info(ticker):
    pos = ticker_list.index(ticker)
    print(f'Ticker: {ticker_list[pos]}')
    print(f'Sector: {sector_list[pos]}')
    print(f'Score: {score_list[pos]}')
    print(f'Coupon: {coupon_list[pos]:.3f} percent')
    print(f'Price: {price_list[pos]:.2f}')
    print(f'Face held: {face_list[pos]:,}')
    print('-' * 30)
```

```
bond_info('QVNE')
Ticker: QVNE
Sector: Energy
Score: 9
Coupon: 5.250 percent
Price: 98.50
Face held: 1,500,000
------------------------------
```

A ticker not held

```
bond_info('ABCD')
ValueError: 'ABCD' is not in list
```

If the layout ever changes, it changes in one place.

Read the last line first, as on Day 3: index() raises a ValueError for a value the list does not hold.

Predict first: three more

Predict 4

"coupon_printed" prints QVNE's coupon and "coupon_returned" returns it. Each result is stored, then printed. What does each cell show?

```
def coupon_printed(ticker):
    pos = ticker_list.index(ticker)
    print(coupon_list[pos])
result = coupon_printed('QVNE')
print(result)
```

```
def coupon_returned(ticker):
    pos = ticker_list.index(ticker)
    return coupon_list[pos]
result = coupon_returned('QVNE')
print(result)
```

Predict 5

Two analysts write a rating bucket with the same tests in a different order. PLWK scores 7. What does each function return?

```
def bucket_one(score):
    if score <= 7:
        label = 'A or better'
    elif score <= 10:
        label = 'BBB'
    else:
        label = 'High yield'
    return label
print('Function 1:', bucket_one(7))
```

```
def bucket_two(score):
    if score <= 10:
        label = 'BBB'
    elif score <= 7:
        label = 'A or better'
    else:
        label = 'High yield'
    return label
print('Function 2:', bucket_two(7))
```

Predict 6

Now each key is a ticker, and its value lists that bond's score, coupon, and price. What does the loop print?

```
by_ticker = {
    'QVNE': [9, 5.25, 98.5],
    'DNMR': [11, 6.125, 102.75],
    'PLWK': [7, 4.75, 95.0],
}
for key, value in by_ticker.items():
    print(key, value[1])
```

5.5 and 5.6 One dictionary, then one table

Six lists, one dictionary

```
portfolio = {
    'Ticker': ticker_list,
    'Sector': sector_list,
    'Score': score_list,
    'Coupon': coupon_list,
    'Price': price_list,
    'Face': face_list,
}
portfolio
```

```
for key, value in portfolio.items():
    print(key, value[3])
Ticker HLVF
Sector Consumer
Score 14
Coupon 7.5
Price 100.5
Face 250000
```

df = pd.DataFrame(portfolio)

Ticker	Sector	Score	Coupon	Price	Face
QVNE	Energy	9	5.250	98.50	1500000
DNMR	Transportation	11	6.125	102.75	500000
PLWK	Materials	7	4.750	95.00	750000
HLVF	Consumer	14	7.500	100.50	250000
OSTV	Telecom	11	5.750	98.50	1000000
VYRH	Health Care	12	6.750	102.00	400000

Excel side by side

A DataFrame is a worksheet: named columns, one bond per row. Here every step that made it is written down, and reruns tomorrow on a new email.

5.7 Every column in one line

```
print(df.shape)
(6, 6)
df.describe()
```

The median score is 11: after the downgrade, the middle of the book is high yield.

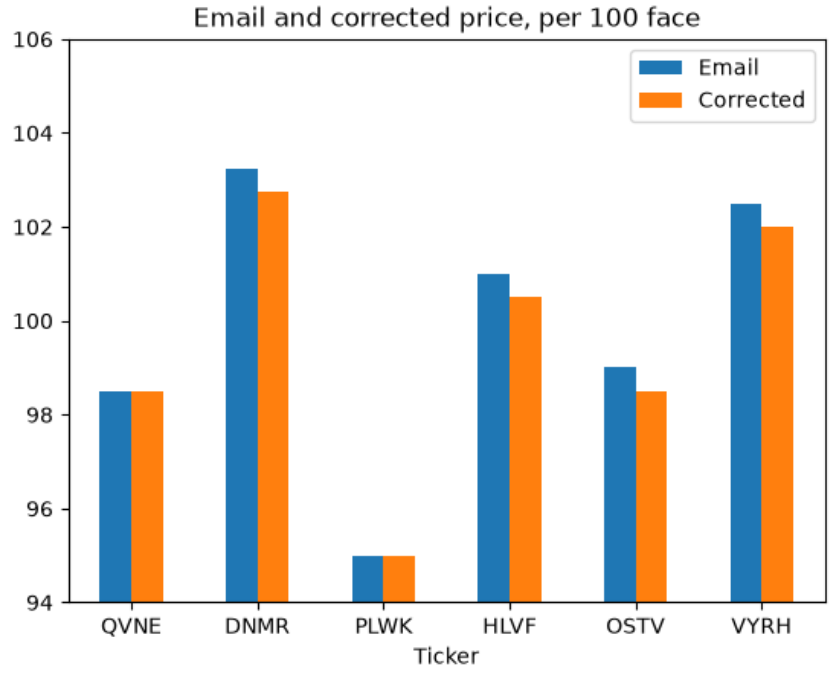
One line, 32 numbers: eight for each of the four number columns.

Excel side by side: each row of describe(), on the corrected prices in I2:I7

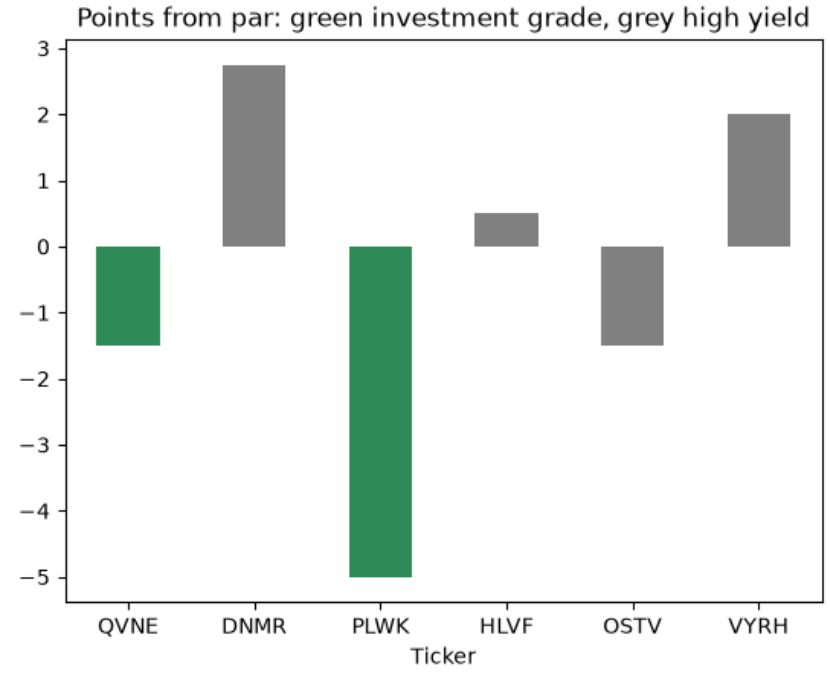
Row	Means	Excel
count	how many values	=COUNT(I2:I7)
mean	the average	=AVERAGE(I2:I7)
std	standard deviation: how spread out the values are	=STDEV.S(I2:I7)
min, max	the two ends	=MIN(I2:I7), =MAX(I2:I7)
25%, 75%	a quarter, and three quarters, of the way up	=QUARTILE.INC(I2:I7,1), =QUARTILE.INC(I2:I7,3)
50%	the median, the middle value	=MEDIAN(I2:I7)

The numbers summarize; a chart shows each bond.

What the correction did, and price against par



Four bars drop half a point; QVNE and PLWK, investment grade, do not.



Three above par, three below; four of six high yield.

Tie the table, then hand it on

```
total_face_after = df['Face'].sum()
assert len(df) == len(ticker_list), 'a bond went missing'
assert total_face_after == total_face_before, 'the face no longer ties'
print(f'Total face held: {total_face_after:,}')
Total face held: 4,400,000
```

```
df.to_csv('handover.csv', index=False)
handover = pd.read_csv('handover.csv')
assert handover['Face'].sum() == total_face_before, 'the file does not tie to the email'
print(len(handover), 'bonds read back from handover.csv')
6 bonds read back from handover.csv
```

Excel side by side

File, Save As, CSV (Comma delimited) does the job of to_csv(); opening the file again is read_csv().

4,400,000 in the email, the lists, the table, and the file: the book is in order.

Summary: the report to the desk

The desk asked	Answer
Put the book in order	one pandas table: six bonds, six columns, nothing missing
Tie it to the email	4,400,000 face, by assert on the lists, the table, and the file
Look up one bond	bond_info('QVNE'): one line
Update a bond	OSTV's downgrade by position; the high yield correction in one loop
Answer simple questions	2 investment grade, 4 high yield, 3 above par
Hand it on for analysis	df, a DataFrame, and a CSV file read back with read_csv()

Next, in Week 2, NumPy does the arithmetic on a whole column of bonds in a single line.